SPACE COLONY

Technical Report — Final Delivery

Nicolás Acero Ladino · Erik Fernandez Rios · Laura Paez Cifuentes

Computer Sciences I · Semester 2026-I

Universidad Distrital Francisco José de Caldas

Eng. Carlos Andrés Sierra, M.Sc.

1. Introduction

Space Colony is a turn-based strategy game in which the human player faces an AI opponent (the machine) on an 8x8 grid representing the surface of a planet. We developed the game as a final project for Computer Science I, and it serves as a unification of the two main pillars of the course: algorithms and data structures.

The project is developed in three C++ responsible for the game management and data structures, Python which contains the machine's algorithms and a Pygame graphical interface that represents the board and accepts input from the player.Having greedy and backtracking algorithms as our main algorithms, we use trees and linked lists, which help us review and demonstrate their functionalities and utilities in a complete project.

2. Game Overview

2.1 Objective

This project seeks the implementation of themes seen throughout the semester such as linked lists and algorithms in a practical implementation that demonstrates the functionality and real use of these in projects through a game with the dynamics of accumulating the greatest amount of resources and controlling the largest possible territory. divided into two: an expansion phase followed by a Conquest phase.

2.2 Expansion Phase

Both the player and the AI take turns placing colonies in empty grid cells. Each cell has a fixed resource value (1–100) and an energy cost equal to resources ÷ 10 (minimum 1). Both start at 150 units of energy, with these prohibitions:

• Adjacency: After the first colony, each new colony must be located within a distance of ≤ 2 from any existing occupied cell.

• Minimum distance: No two colonies belonging to the same owner can be closer than the distance of 2 from each other.

• Energy budget: A colony can only be placed if the owner can afford the cost of energy.

This ends when the grid is completely full, neither player can expand, or both players pass in a row.

2.3 Conquest Phase

Expansions end, players attack adjacent enemy grids (distance = 1). The combat uses a model:

```
attacker_power = attacker_resources + Random(1, 6)
defender_power = defender_resources + Random(1, 6)

If attacker_power > defender_power  →  attacker captures the cell
```

```
Otherwise                          →  defender keeps the cell
```

The game ends when any player loses all the cells or the turn limit is reached. The winner is whoever has the most resources.

2.4 Roles

Role	Decisions	Mechanism
Player	Manual — clicks on grid cells	Pygame mouse events
AI	Automatic — greedy expansion, greedy attack	Python algorithms + C++ engine

3. Algorithms

3.1 Greedy Algorithm

3.1.1 Concept

A greedy algorithm gives a solution step by step, choosing the option that currently looks best without considering past decisions. It does not explore alternatives or guarantee an optimal overall result, but it produces good solutions.

3.1.2 Application in Space Colony

The greedy strategy is used in two different contexts:

• Expansion (greedy_select_move): The AI searches for each affordable adjacent empty cell and chooses the one with the highest resource value.

• Combat (greedy_attack): The AI scans each enemy cell adjacent to any of its own colonies and attacks the one with the lowest resource value, seeking to maximize power.

3.1.3 Pseudocode

GREEDY_SELECT_MOVE — Expansion Phase:

```
GREEDY_SELECT_MOVE(available_cells, energy):
  best_cell  ← null
  max_value  ← −∞
  for each cell in available_cells:
    if cell.energy_cost ≤ energy:
      value ← cell.resources
      if value > max_value:
        max_value ← value
        best_cell ← cell
  return best_cell
```

GREEDY_ATTACK — Conquest Phase:

```
GREEDY_ATTACK(neighbor_enemy_cells):
  target         ← null
  lowest_defense ← +∞
  for each cell in neighbor_enemy_cells:
```

```
    defense ← cell.resources
    if defense < lowest_defense:
      lowest_defense ← defense
      target         ← cell
  return target
```

3.1.4 Complexity Analysis

Function	Input size n	Time	Space
greedy_select_move	available cells	O(n)	O(1)
greedy_attack	enemy neighbours	O(n)	O(1)
get_ai_expansion_candidates	all cells (64)	$O(G^2)$	O(k)
get_ai_attack_candidates	all cells (64)	$O(G^2)$	O(k)

3.2 Backtracking Algorithm

3.2.1 Concept

Backtracking is a systematic algorithm that incrementally constructs candidate solutions and abandons (backtracks) a candidate as soon as it is determined that the candidate cannot lead to a valid or optimal solution. Unlike greed, backtracking can find globally optimal solutions within a finite search space. Its main cost is the exponential time in the worst case, which must be controlled by pruning strategies.

3.2.2 Application in Space Colony

During the Expansion Phase, the AI uses backtracking to plan ahead and find the sequence of colonizations that maximizes the total resources accumulated while respecting three strict constraints:

• Distance constraint: Each new colony must maintain at least Manhattan 2 distance from all existing AI colonies (satisfies_distance).

• Energy budget: the energy cost of each step must not exceed the remaining budget.

• Adjacency rule: each new cell must be reachable from the current border (Manhattan distance is exactly 2 from the border cell).Since the entire search space is exponential, two pruning strategies are applied:

• Greedy sorting: Candidate cells are sorted in descending order by resource value before recursion, so high-value paths are explored first and better upper bounds are set early, allowing weaker branches to be pruned earlier.

In each turn only the first cell of the best sequence found is executed, since the game engine processes one action per turn.

3.2.3 Pseudocode

```
BACKTRACK(current_pos, visited, total_resources, energy, depth):
  if total_resources > best_solution.score:
    best_solution ← (total_resources, copy(sequence))
```

```
  if depth ≥ MAX_DEPTH:
    return                            // depth-limit pruning

  candidates ← GET_NEIGHBORS(current_pos)          // distance=2 ring
  sort candidates descending by resources           // greedy ordering

  for each neighbor in candidates:
    if neighbor ∈ visited:   continue
    cost ← max(1, neighbor.resources / 10)
    if cost > energy:        continue                // budget pruning
    if NOT satisfies_distance(neighbor, visited):   continue

    mark neighbor as visited
    append neighbor to sequence
    BACKTRACK(neighbor, visited,
              total_resources + neighbor.resources,
              energy − cost, depth + 1)
    remove neighbor from sequence   // undo (backtrack)
    unmark neighbor

satisfies_distance(pos, existing):
  for each colony in existing:
    if manhattan(pos, colony) < MIN_DISTANCE:
      return false
  return true
```

3.2.4 Complexity Analysis

Aspect	Value	Complexity	Note
Max deph = 5	b ≤ 12	$O(12^5) \approx O(248{,}832)$	Practical upper bound
satisfies_distance check	k colonies	O(k)	Per candidate
Space (call stack)	depth ≤ 5	O(d) = O(5)	Constant in practice

4. Data Structures (C++)

Both data structures are implemented entirely from scratch using raw C++ pointers (new / delete). No STL containers (std::list, std::map, std::set) were used for the required structures, in compliance with the project constraints.

4.1 Singly Linked List — Settlement History

4.1.1 Purpose

The linked list stores the complete chronological log of every colonisatio and capture event. It is used by the C++ engine to: (1) validate the distance constraint during expansion by traversing all colonies belonging to a giveen owner; (2) reconstruct the colonies array that is serialised into state.json at the end of every turn.

4.1.2 Node Structure

```
struct Node {
  int         x;          // column (0-7)
  int         y;          // row    (0-7)
  std::string owner;      // "player" or "ai"
  int         resources;  // resource value at time of colonisation
  Node*       next;       // pointer to next event
};

class LinkedList {
  Node* head;   // pointer to first node
  Node* tail;   // pointer to last node (O(1) append)
  int   count;  // cached node count
public:
  void        append(int x, int y, const string& owner, int resources);
  void        remove_last();
  void        print_all()  const;
  int         size()       const;
  const Node* get_head()   const;
  void        clear();
};
```

The list maintains both a head and a tail pointer. The tail pointer allows O(1) insertion at the end, which is the most frequent operation (one append per colonisation event). The head pointer enables O(n) forward traversal, used during distance validation.

4.1.3 Operation Complexity

Operation	Time	Space	Notes
append()	O(1)	O(1)	Tail pointer avoids traversal
remove_last()	O(n)	O(1)	Must walk to second-to-last node
get_head() / traversal	O(n)	O(1)	Used for distance checks
size()	O(1)	O(1)	Maintained as a counter
clear() / destructor	O(n)	O(1)	Must free every heap node

4.2 AVL Tree — Resource Deposit Registry

4.2.1 Purpose

The AVL tree stores each active colony sorted by resource value. It gives the AI instant access to the colony of highest or lowest value to any owner, driving greedy expansion like combat targeting. Because the C++ engine rebuilds the tree from the colony array on each launch, it must also support efficient deletion (when a cell is captured in combat).

4.2.2 Why AVL and Not BST

A simple binary search tree (BST) is set to O(n) height when elements are inserted in sorted or quasi-sorted order. During the initial expansion, the greedy AI algorithm tends to colonize cells in descending order. The AVL tree after every insertion and deletion is updated ensuring O(log n) height at all times.

4.2.3 Node Structure

```
struct AVLNode {
  int         resources;    // primary ordering key
  int         energy_cost;
  int         x, y;         // position — used to break ties
  std::string owner;        // "player" or "ai"
  AVLNode*    left;
  AVLNode*    right;
  int         height;       // height of subtree rooted here
};

// Composite key used for unique ordering:
static int colony_key(int resources, int x, int y) {
  return resources * 100 + (x * 8 + y);
}
```

4.2.4 Rotation Cases

```
Balance factor = height(left subtree) − height(right subtree)
Valid range:  {−1, 0, 1}  →  tree is balanced

Case            Condition                          Fix
──────────────────────────────────────────────────────────────────
Left-Left       balance > 1,  left child ≥ 0      rotate_right(node)
Left-Right      balance > 1,  left child < 0      rotate_left(left)
                                                   rotate_right(node)
Right-Right     balance < −1, right child ≤ 0     rotate_left(node)
Right-Left      balance < −1, right child > 0     rotate_right(right)
                                                   rotate_left(node)
```

4.2.5 Operation Complexity

Operation	Time	Space	Notes
insert()	O(log n)	O(log n)	Recursive; stack depth = height
remove()	O(log n)	O(log n)	In-order successor used for 2-child case
find_max() / find_min()	O(log n)	O(1)	Rightmost / leftmost node traversal
find_max_by_owner()	O(n)	O(log n)	Owner not part of key; full traversal needed
find_min_by_owner()	O(n)	O(log n)	Same reason as find_max_by_owner
rebalance() / rotation	O(1)	O(1)	Constant pointer reassignments per call
clear() / destructor	O(n)	O(log n)	Post-order traversal frees every node

5. JSON Bridge — Python / C++ Communication

5.1 Architecture

The Python and C++ layers are separate processes that never share memory. They communicate exclusively through two plain-text JSON filess in the data/ directory:

- `input.json` written by Python (bridge.py) before invoking the engine; contains the action to be processed.
- `state.json` written by the C++ engine after processsing the action; contains the complete updated game state for the UI to render.

The Python bridge (bridge.py) invokes the compiled C++ binary as a child process using subprocess.run() and waits synchronously for it to finish before reading state.json. This sequential, blocking protocol ensures that the UI never reads a partial or stale state.

5.2 Input JSON Schema

Python writes one of four action objects to input.json:

```
// Colonise a cell (Expansion Phase)
{
  "action"      : "colonize",
  "destination" : { "x": <int 0-7>, "y": <int 0-7> }
}

// Attack an enemy cell (Conquest Phase)
{
  "action"      : "attack",
  "origin"      : { "x": <int>, "y": <int> },
  "destination" : { "x": <int>, "y": <int> }
}

// Skip turn
{ "action": "pass" }

// Full game reset
{ "action": "reset" }
```

5.3 State JSON Schema

The C++ engine writes the full game state to state.json after every action:

```
{
  "phase"                 : "expansion" | "combat",
  "turn"                  : "player" | "ai",
  "player_total_resources": <int>,
  "ai_total_resources"    : <int>,
  "player_energy"         : <int>,
  "ai_energy"             : <int>,
  "current_turn"          : <int>,
  "max_turns"             : 200,
  "game_over"             : <bool>,
  "winner"                : "" | "player" | "ai" | "draw",
  "last_result"           : <string>,
  "grid"       : [[<int> × 8] × 8],   // resource values
  "owner_grid" : [[0|1|2  × 8] × 8],  // 0=empty 1=player 2=ai
  "colonies"   : [
    { "x":<int>, "y":<int>, "owner":<str>,
```

```
      "resources":<int>, "defense":<int> },
    ...
  ]
}
```

5.4 Turn Sequence

```
1. main.py  detects player click or invokes AI algorithm
2. bridge.write_input(action_dict)  →  writes input.json
3. subprocess.run([engine_binary, data_dir])  →  engine executes
4. engine reads input.json, processes action, updates linked list + AVL tree
5. engine writes state.json
6. bridge.read_state()  →  Python reads state.json
7. main.py re-renders the grid from the new state
```

5.5 Auto-Compilation

bridge.py includes an automatic compilation step: if C++ is missing or if any source file (.cpp / .h) is newer than the binary, init_engine() recompiles before the first turn. This means that you never need to run a separate build command: the game always runs the latest engine code.

6. System Architecture

6.1 Folder Structure

```
project/
├── engine/
│   ├── main.cpp          C++ game engine entry point
│   ├── linked_list.cpp   Singly linked list implementation
│   ├── linked_list.h
│   ├── tree.cpp          AVL tree implementation
│   ├── tree.h
│   ├── json.hpp          nlohmann/json header-only library
│   └── engine.exe        compiled binary (Windows)
├── game/
│   ├── algorithms/
│   │   ├── greedy.py     Greedy expansion and attack
│   │   └── backtracking.py   Backtracking planner
│   └── ui/
│       ├── main.py       Pygame UI and game loop
│       └── bridge.py     JSON file I/O + engine invocation
└── data/
    ├── input.json        Python → C++
    └── state.json        C++ → Python
```

6.2 Component Responsibilities

Component	Language	Responsibilities
main.cpp	C++	Game-state management, action processing, phase transitions, winner determination, JSON I/O
linked_list.cpp	C++	Colonisation history; distance validation; state serialisation to colonies array

tree.cpp	C++	Resource-ordered colony registry; fast max/min lookups; updated on every colonise and capture event
greedy.py	Python	Expansion candidate generation and selection; attack candidate generation and target selection
backtracking.py	Python	Multi-step expansion planning with constraint checking and depth-limited search
main.py	Python	Pygame grid rendering (8×8), HUD, player mouse input, game loop, AI turn orchestration
bridge.py	Python	Writes input.json, invokes engine binary, reads state.json; handles auto-compilation

6.3 Work Distribution

Team Member	Role	Deliverables
Nicolás Acero Ladino	Algorithms	greedy.py, backtracking.py, algorithm integration with bridge layer
Laura Paez Cifuentes	C++ Engine	linked_list.cpp/h, tree.cpp/h, main.cpp engine, JSON I/O, compilation
Erik Fernandez Rios	UI / Bridge	main.py (Pygame grid, HUD, game loop), bridge.py (file I/O, auto-compile)

7. Design Decisions, Challenges, and Lessons Learned

7.1 Design Decisions

The decision to use an AVL tree was used by the game pattern. The greedy algorithm colonizes cells in descending order of resources during expansion. AVL tree self-balancing guarantees O(log n) regardless of insertion order.JSON File Bridge over Sockets or Bindings

Three integration approaches were considered: Python-C++ bindings (ctypes/pybind11), inter-process sockets, and plain JSON files. File I/O was chosen for three reasons: (1) it imposes no additional dependencies beyond the nlohmann/json header; (2) the state file can be opened in any text editor for instant debugging; (3) it matches the project specification, which explicitly requires JSON file I/O and prohibits sockets and bindings. The blocking subprocess.run() call proved sufficient because the engine completes each action in milliseconds.

Energy Cost Formula

The formula `cost = max(1, resources / 10)` was designed so that high-resource cells are more expensive to colonise, creating a meaningful trade-off between resource gain and energy sustainability. A minimum cost of 1 prevents free colonisation of very low-resource cells and keeps the energy budget relevant throughout the game.

7.2 Lessons Learned

• Clear separation of responsibilities between languages (C++ for data-intensive computing, Python for higher-level logic and UI) simplified the process independently, a bug in the greedy algorithm could be isolated in greedy.py without touching any C++ code.

• The JSON between layers acted as a feature for clear interface design. Any change to the game state required updating both C++ and Python, which made this implementation necessary.

• The implementation of data structures deepened the understanding of memory management, pointers, and the actual cost of operations.

• The trade-off between quality and real-time performance in the rollback algorithm is an example of the time-space-quality trade-off.

8. Conclusion

Space Colony allowed us to integrate the Computer Science I topics into a single project. The greedy algorithm manages fast and competitive AI decisions both during expansion and in combat. The AVL tree guarantees searches in logarithmic time. The linked list provides an orderly and efficient record of each colonization event.

This project allowed us to go further, developing automatic phase detection, a switching mechanism to handle locked boards among other things, being a very great practical reinforcement of this subject, linking a lot of knowledge and things that not only serve us currently but also for the rest of our semester.